GUÍA DE ESTUDIO PASO A PASO

🍕 Curso Práctico: React y Context API desde Cero

Aprende a construir "La Pizzería Mágica" explicada con peras y manzanas: instalación con Vite, Bootstrap, diagramas de flujo, pseudocódigo y console.log educativos.

🧠 Introducción: ¿Qué es el "Context API" con peras y manzanas?

🍎 La Analogía del Colegio y las Pizzas

Imagina que estás en el 1° piso de tu colegio y quieres darle una caja de pizzas a tu mejor amigo que está en el 5° piso:

- **Sin Context API (La forma difícil / Prop Drilling):** Tendrías que darle la pizza a un niño del 2° piso, ese niño se la pasa a uno del 3°, ese al del 4°, hasta llegar al 5° piso. ¡Qué cansado!
- **Con Context API (La forma mágica):** Es como instalar una **Nube Mágica** en el techo del colegio. Tú subes las pizzas a esa nube, y cualquier niño de cualquier piso solo estira la mano hacia arriba y saca su pedazo de pizza directamente, sin molestar a los demás pisos.

📊 Mapa Mental General (Diagrama de Flujo)



🛠️ PASO 1: Preparando nuestra Cocina (Instalación y Dependencias)

Antes de hacer pizzas, necesitamos construir el edificio de la pizzería (con **Vite**) y comprar las mesas y decoraciones bonitas (con **Bootstrap**).

1. Crear el proyecto en la Terminal de VS Code

Abre la terminal en VS Code (`Terminal → New Terminal`) y ejecuta el comando del robot constructor Vite:

```
npm create vite@latest mi-pizzeria -- --template react
```

- `npm create vite@latest` : Llama al robot constructor en su versión más reciente.
- `mi-pizzeria` : Es el nombre de la carpeta que se va a crear.
- `-- --template react` : Le indica que queremos usar **React**.

2. Entrar a la carpeta e instalar las herramientas

Ejecuta estos 3 comandos uno por uno en la terminal:

```
cd mi-pizzeria
npm install
npm install bootstrap
```

- `cd mi-pizzeria` : Abre la puerta y entra a la carpeta de nuestra tienda.
- `npm install` : Instala el motor y los cables internos de React.
- `npm install bootstrap` : Descarga los estilos de botones, colores y tarjetas bonitas.

3. Conectar Bootstrap en `src/main.jsx`

Abre el archivo `src/main.jsx` y agrega la línea de Bootstrap para encender las luces de diseño en toda la aplicación:

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
// 👇 AGREGA ESTA LÍNEA AQUÍ: Enciende los estilos de Bootstrap
import 'bootstrap/dist/css/bootstrap.min.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

4. Encender el servidor en vivo

```
npm run dev
```

Haz clic con `Ctrl + Clic` (o `Cmd + Clic`) sobre el enlace `http://localhost:5173/` para abrir tu pizzería en el navegador.

☁️ PASO 2: Creando la Nube Mágica (El Contexto)

> 📡 **¿Cómo funciona la Nube por dentro?**
>
> Para crear un Contexto en React siempre necesitamos dos piezas que trabajan como una estación de radio:
>
> 1. **La Señal de Radio (`createContext`):** Es la caja o canal donde viajarán los datos.
> 2. **La Antena Transmisora (`Provider`):** Es un componente especial que **abraza (envuelve)** nuestra tienda. Todo lo que esté adentro de este abrazo podrá escuchar la señal.

📝 Pseudocódigo del Paso 2

```
1. Crear la caja mágica llamada "PizzaContext".
2. Crear la Antena llamada "PizzaProvider".
3. Adentro de la Antena, guardar el Menú de Pizzas (Margarita, Pepperoni, Hawaiana).
4. Adentro de la Antena, crear una canasta vacía para el Carrito = [].
5. Imprimir un console.log() para comprobar que la Antena se encendió.
6. Compartir el Menú y el Carrito con toda la tienda.
```

📊 Diagrama de Flujo del Paso 2



1. Código de `src/context/PizzaContext.jsx`

Crea la carpeta `src/context` y adentro el archivo `PizzaContext.jsx`:

```
import { createContext, useState } from 'react';

// 1. ¡Aquí nace la Nube Mágica!
export const PizzaContext = createContext();

// 2. Creamos la "Antena Transmisora" (El Provider).
// "{ children }" representa a todas las pantallas que meteremos dentro de este abrazo.
export const PizzaProvider = ({ children }) => {

  // 📋 DATO 1: Nuestro menú de pizzas
  const menuPizzas = [
    { id: 1, nombre: 'Pizza Margarita', precio: 10, emoji: '🧀' },
    { id: 2, nombre: 'Pizza Pepperoni', precio: 12, emoji: '🍕' },
    { id: 3, nombre: 'Pizza Hawaiana', precio: 11, emoji: '🍍' },
  ];

  // 🛒 DATO 2: Nuestro carrito de compras (empieza como lista vacía [])
  const [carrito, setCarrito] = useState([]);

  // 🔍 CONSOLE.LOG EDUCATIVO #1:
  console.log("☁️ [NUBE MÁGICA ACTIVA] Menú disponible:", menuPizzas);
  console.log("🛒 [ESTADO DEL CARRITO]:", carrito);

  // 3. En "value={{ menuPizzas, carrito }}" metemos la bolsita de regalos para la tienda
  return (
    <PizzaContext.Provider value={{ menuPizzas, carrito }}>
      {children}
    </PizzaContext.Provider>
  );
};
```

2. Conectar la Antena en `src/App.jsx`

```
import { PizzaProvider } from './context/PizzaContext';

function App() {
  return (
    <PizzaProvider>
      <div className="container mt-5 text-center">
        <h1 className="display-4 fw-bold text-danger">🍕 La Pizzería Mágica</h1>
        <p className="lead text-secondary">
          ¡Nuestra nube mágica (Context) ya está encendida! Mira la consola.
        </p>
      </div>
    </PizzaProvider>
  );
}

export default App;
```

📻 PASO 3: Bajando las Pizzas a la Pantalla (`useContext`)

> **📻 ¿Qué es useContext? (El Walkie-Talkie Receptor)**
>
> Si en el Paso 2 creamos la Antena Transmisora (`PizzaProvider`), ahora necesitamos un **Walkie-Talkie receptor** para escuchar esa señal desde cualquier pantalla. En React, ese Walkie-Talkie se llama `useContext(PizzaContext)` .

📊 Diagrama de Flujo del Paso 3



1. Crear `src/components/Catalogo.jsx`

Crea la carpeta `src/components` y adentro el archivo `Catalogo.jsx` :

```
import { useContext } from 'react';
import { PizzaContext } from '../context/PizzaContext';

export const Catalogo = () => {
  // 🗃️ ¡AQUÍ ESTÁ LA MAGIA! Sacamos "menuPizzas" directamente de la nube
  const { menuPizzas } = useContext(PizzaContext);

  // 🔍 CONSOLE.LOG EDUCATIVO #2:
  console.log("🗃️ [CATÁLOGO] ¡Señal recibida desde la nube! Pizzas:", menuPizzas);

  return (
    <div className="row mt-4">
      {menuPizzas.map((pizza) => (
        <div className="col-md-4 mb-3" key={pizza.id}>
          <div className="card shadow-sm h-100 border-danger">
            <div className="card-body text-center">
              <div className="display-1 mb-2">{pizza.emoji}</div>
              <h5 className="card-title fw-bold">{pizza.nombre}</h5>
              <p className="card-text fs-4 text-success fw-bold">${pizza.precio}</p>
              <button className="btn btn-danger w-100">
                🛒 Agregar al carrito
              </button>
            </div>
          </div>
        </div>
      ))}
    </div>
  );
};
```

💡 **¿Por qué usamos `key={pizza.id}`?**

Para que React no se confunda de cuál tarjeta es cuál cuando dibuja una lista con `.map()`, le ponemos un "collar con número de identificación único" (`key`) a cada elemento.

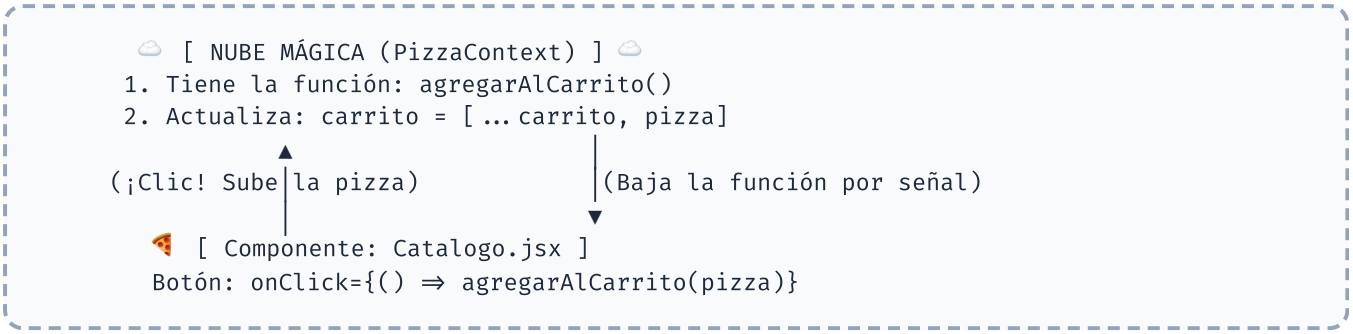
🚀 PASO 4: Dándole Vida al Botón de Comprar

🧺 **El secreto de los tres puntitos `...` (Spread Operator)**

En React no podemos empujar una pizza nueva a la fuerza dentro de la canasta vieja. Tenemos que crear una canasta nueva copiando lo que ya había antes y poniendo la pizza nueva al lado: `setCarrito([ ...carrito, pizzaElegida])`.

- `...carrito` significa: *"Saca todas las pizzas que ya estaban en el carrito viejo y ponlas aquí"*.
- `, pizzaElegida` significa: *"...y agrégale esta pizza nueva al final"*.

📊 Diagrama de Flujo del Paso 4 (Viaje de Ida y Vuelta)



1. Agregar la función en `src/context/PizzaContext.jsx`

```
  // 👇 Nuestro robot ayudante en la nube para meter pizzas al carrito
  const agregarAlCarrito = (pizzaElegida) ⇒ {
    // 🔍 CONSOLE.LOG EDUCATIVO #3:
    console.log("🚀 [ACCIÓN] ¡El usuario hizo clic! Subiendo a la nube:", pizzaElegida.nombre
    setCarrito([ ... carrito, pizzaElegida]);
  };

  // No olvides incluir agregarAlCarrito dentro del value del Provider:
  // <PizzaContext.Provider value={{ menuPizzas, carrito, agregarAlCarrito }}>
```

2. Conectar el botón en `src/components/Catalogo.jsx`

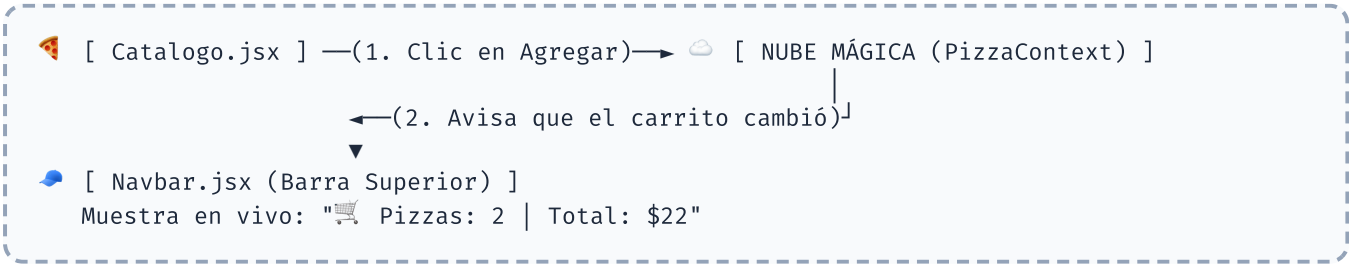
```
export const Catalogo = () ⇒ {
  // 👇 Ahora sacamos DOS cosas de la nube: el menú Y la función agregarAlCarrito
  const { menuPizzas, agregarAlCarrito } = useContext(PizzaContext);
  // ...
              <button
                className="btn btn-danger w-100"
                onClick={() ⇒ agregarAlCarrito(pizza)}
              >
                🛒 Agregar al carrito
              </button>
```

> 🤔 **¿Por qué escribimos `onClick={() ⇒ agregarAlCarrito(pizza)}` con la flechita `() ⇒` ?**
>
> Si no pusiéramos la flechita `() ⇒`, React ejecutaría la función apenas cargue la página sin esperar tu clic. La flechita le dice: *"¡Espera a que el humano haga clic!"*.

🧢 PASO 5: Creando la Barra Superior Inteligente (`Navbar.jsx`)

Ahora crearemos un segundo componente totalmente separado que escucha la misma nube y calcula cuántas pizzas llevamos (con `carrito.length`) y cuánto dinero suman (con `.reduce()`).

📊 Diagrama de Flujo del Paso 5



Código de `src/components/Navbar.jsx`

```
import { useContext } from 'react';
import { PizzaContext } from '../context/PizzaContext';

export const Navbar = () => {
  // 📻 Sintonizamos la nube y sacamos "carrito"
  const { carrito } = useContext(PizzaContext);

  // 🧮 CAJA REGISTRADORA: empieza en 0 y suma el precio de cada pizza
  const totalPagar = carrito.reduce((acumulador, pizza) => acumulador + pizza.precio, 0);

  // 🔍 CONSOLE.LOG EDUCATIVO #4:
  console.log(`🧢 [NAVBAR] Actualizado → Cantidad: ${carrito.length} pizzas | Total: $${tota

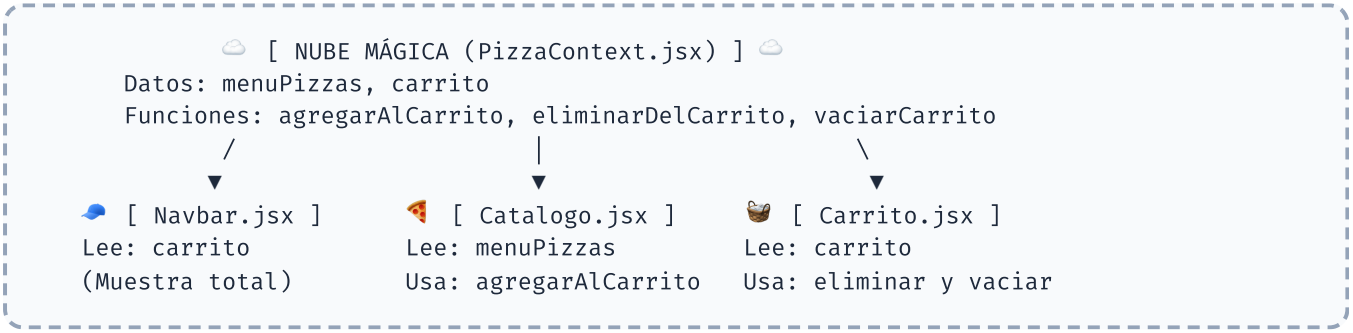
  return (
    <nav className="navbar navbar-dark bg-dark shadow-sm mb-4 p-3">
      <div className="container d-flex justify-content-between align-items-center">
        <span className="navbar-brand fw-bold fs-4">🍕 Pizzería Don Contexto</span>
        <div className="bg-warning text-dark px-3 py-2 rounded-pill fw-bold">
          🛒 Pizzas: <span className="badge bg-danger">{carrito.length}</span>
          {' | '}
          Total: ${totalPagar}
        </div>
      </div>
    </nav>
  );
};
```

🧺 PASO 6: El Panel de Pedidos, Eliminar y Vaciar (Gran Final)

🗑️ ¿Cómo borramos una pizza con `.filter()`?

En programación las sillas de una fila empiezan a contarse desde el **0** (silla 0, silla 1, silla 2...). Para quitar una pizza usamos el colador mágico `.filter()`: le decimos que deje pasar a la nueva canasta a todas las pizzas **EXCEPTO** la que esté sentada en la posición que queremos borrar (`index ≠ indiceABorrar`).

📊 Arquitectura Completa de tu Aplicación



1. Versión Final de `src/context/PizzaContext.jsx`

```
import { createContext, useState } from 'react';

export const PizzaContext = createContext();

export const PizzaProvider = ({ children }) => {
  const menuPizzas = [
    { id: 1, nombre: 'Pizza Margarita', precio: 10, emoji: '🧀' },
    { id: 2, nombre: 'Pizza Pepperoni', precio: 12, emoji: '🍕' },
    { id: 3, nombre: 'Pizza Hawaiana', precio: 11, emoji: '🍍' },
  ];

  const [carrito, setCarrito] = useState([]);

  // Superpoder 1: Agregar una pizza
  const agregarAlCarrito = (pizzaElegida) => {
    console.log("🚀 [ACCIÓN] Subiendo a la nube:", pizzaElegida.nombre);
    setCarrito([...carrito, pizzaElegida]);
  };

  // Superpoder 2: Eliminar una pizza según su posición (índice)
  const eliminarDelCarrito = (indiceABorrar) => {
    console.log("🗑️ [ACCIÓN] Quitando la pizza en la posición:", indiceABorrar);
    const nuevoCarrito = carrito.filter((pizza, index) => index !== indiceABorrar);
    setCarrito(nuevoCarrito);
  };

  // Superpoder 3: Vaciar todo el carrito
  const vaciarCarrito = () => {
    console.log("🧹 [ACCIÓN] ¡Vaciando todo el carrito en la nube!");
    setCarrito([]);
  };

  console.log("🛒 [ESTADO DEL CARRITO ACTUAL]:", carrito);

  return (
    <PizzaContext.Provider
      value={{ menuPizzas, carrito, agregarAlCarrito, eliminarDelCarrito, vaciarCarrito }}
    >
      {children}
    </PizzaContext.Provider>
  );
};
```

2. Código Completo de `src/components/Carrito.jsx`

```
import { useContext } from 'react';
import { PizzaContext } from '../context/PizzaContext';

export const Carrito = () => {
  const { carrito, eliminarDelCarrito, vaciarCarrito } = useContext(PizzaContext);

  // 🔍 CONSOLE.LOG EDUCATIVO #5:
  console.log("🛒 [COMPONENTE CARRITO] Renderizando lista con", carrito.length, "elementos.")

  return (
    <div className="card mt-5 mb-5 shadow border-dark">
      <div className="card-header bg-dark text-white d-flex justify-content-between align-ite
        <h4 className="mb-0">🛒 Detalle de tu Pedido</h4>
        {carrito.length > 0 && (
          <button className="btn btn-outline-warning btn-sm" onClick={vaciarCarrito}>
            🧹 Vaciar Carrito
          </button>
        )}
      </div>
      <div className="card-body">
        {carrito.length === 0 ? (
          <p className="text-muted mb-0 py-3">
            Tu carrito está vacío 🥺 ¡Agrega unas pizzas arriba!
          </p>
        ) : (
          <ul className="list-group">
            {carrito.map((pizza, index) => (
              <li key={index} className="list-group-item d-flex justify-content-between align
                <span className="fs-5">
                  {pizza.emoji} <strong>{pizza.nombre}</strong> — <span className="text-succe
                </span>
                <button
                  className="btn btn-outline-danger btn-sm"
                  onClick={() => eliminarDelCarrito(index)}
                >
                  ❌ Quitar
                </button>
              </li>
            ))}
          </ul>
        )}
      </div>
    </div>
  );
};
```

3. Archivo Principal Final `src/App.jsx`

```
import { PizzaProvider } from './context/PizzaContext';
import { Catalogo } from './components/Catalogo';
import { Navbar } from './components/Navbar';
import { Carrito } from './components/Carrito';

function App() {
  return (
    <PizzaProvider>
      <Navbar />
      <div className="container mt-4 text-center">
        <h1 className="display-5 fw-bold text-danger">🍕 Menú de Pizzas</h1>
        <p className="lead text-secondary">
          ¡Tu tienda con Context API está 100% conectada!
        </p>
        <Catalogo />
        <Carrito />
      </div>
    </PizzaProvider>
  );
}

export default App;
```

🎓 Resumen de Graduación: Las 3 Reglas de Oro de Context API

1. `createContext()` **(La Nube):** Fabrica el canal donde vivirán tus datos globales.
2. `<PizzaProvider value={{ ... }}>` **(La Antena):** Envuelve tu aplicación en `App.jsx` y comparte en la bolsita `value` todos los datos y funciones que quieras regalar.
3. `useContext(PizzaContext)` **(El Walkie-Talkie):** Úsalo dentro de cualquier componente (`Navbar`, `Catalogo`, `Carrito`) para leer o modificar la nube al instante y sin cables.